

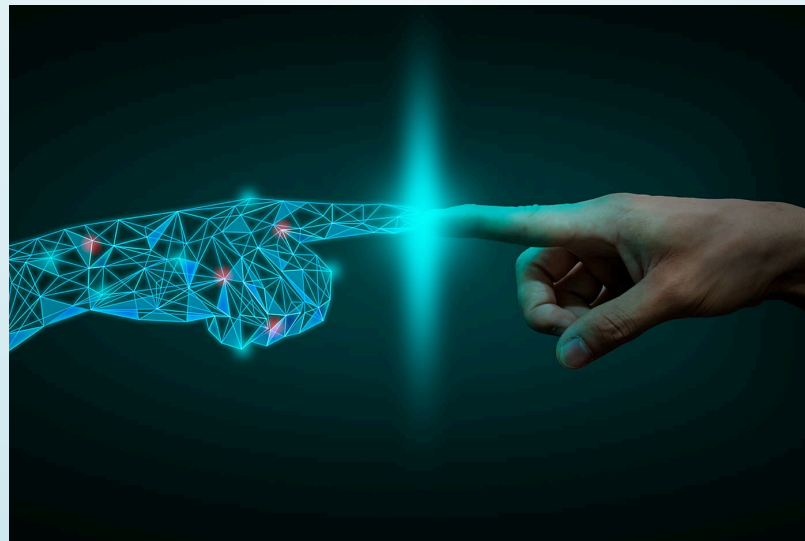
ARTIFICIAL NEURAL NETWORKS

Neural Networks, Deep Learning and
Artificial Intelligence

FROM ARTIFICIAL NEURAL NETWORKS TO ARTIFICIAL INTELLIGENCE

HISTORICAL BACKGROUND (1)

- In the post-pandemic world, a lightning rise of AI, with a mess of realities and promises is impacting society.
- Since the emergence -and rapid growth- of Large Language Models and Generative AI everybody has an experience, an opinion, or a fear on the topic.



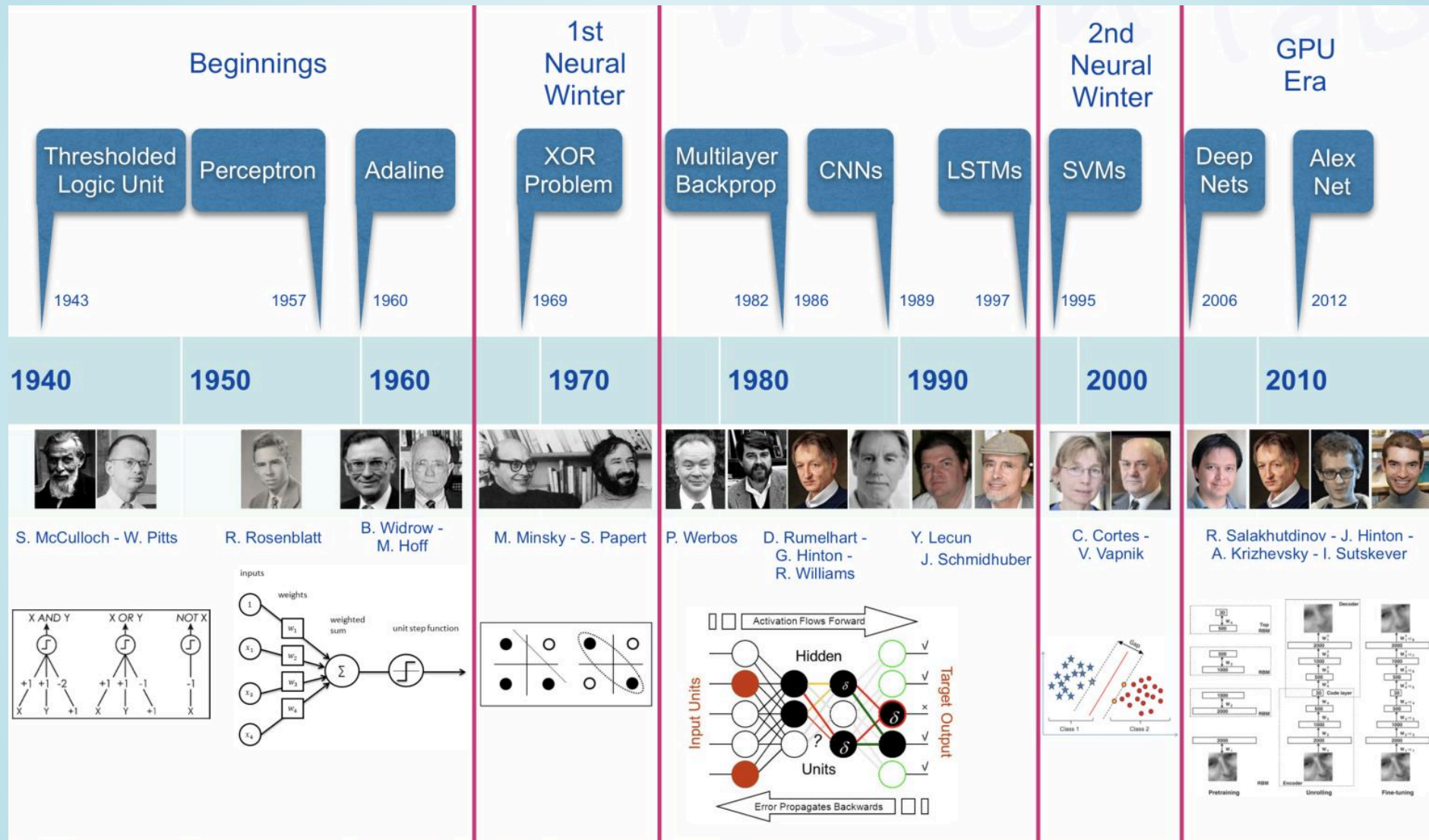
IS IT JUST MACHINE LEARNING?

- Many tasks performed by AI can be described as **Predictive** as seen in *Recommendation systems, Image recognition or Natural language processing*.
- However, AI is broader than just prediction systems
- **Generative AI** is also able to generate new content.
- Both, predictive and generative capabilities of AI have far-reaching implications beyond technologies, including ethical or social aspects.

AI, ANNS AND DEEP LEARNING

- In many contexts, talking about AI means talking about *Deep Learning (DL)*.
- DL is the dominant paradigm in modern (2026) AI and is behind applications such as *self-driving cars, voice assistants, and medical diagnosis systems*.
- DL originates in the field of *Artificial Neural Networks*
- DL extends the basic principles of ANNs by:
 - Adding complex architectures and algorithms and
 - Enabling scalable learning from large data

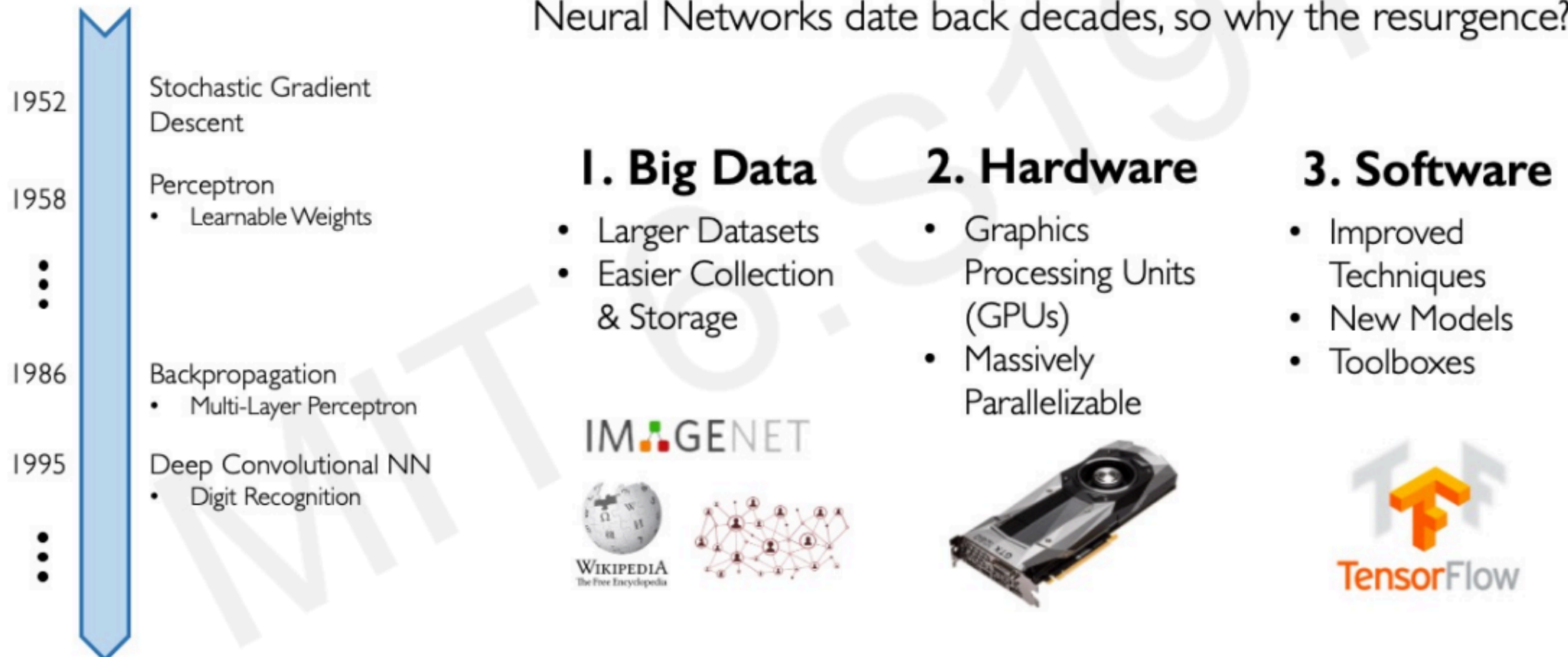
THE EARLY HISTORY OF AI (1)



A Quick History of AI, ML and DL

FROM ANN TO DEEP LEARNING

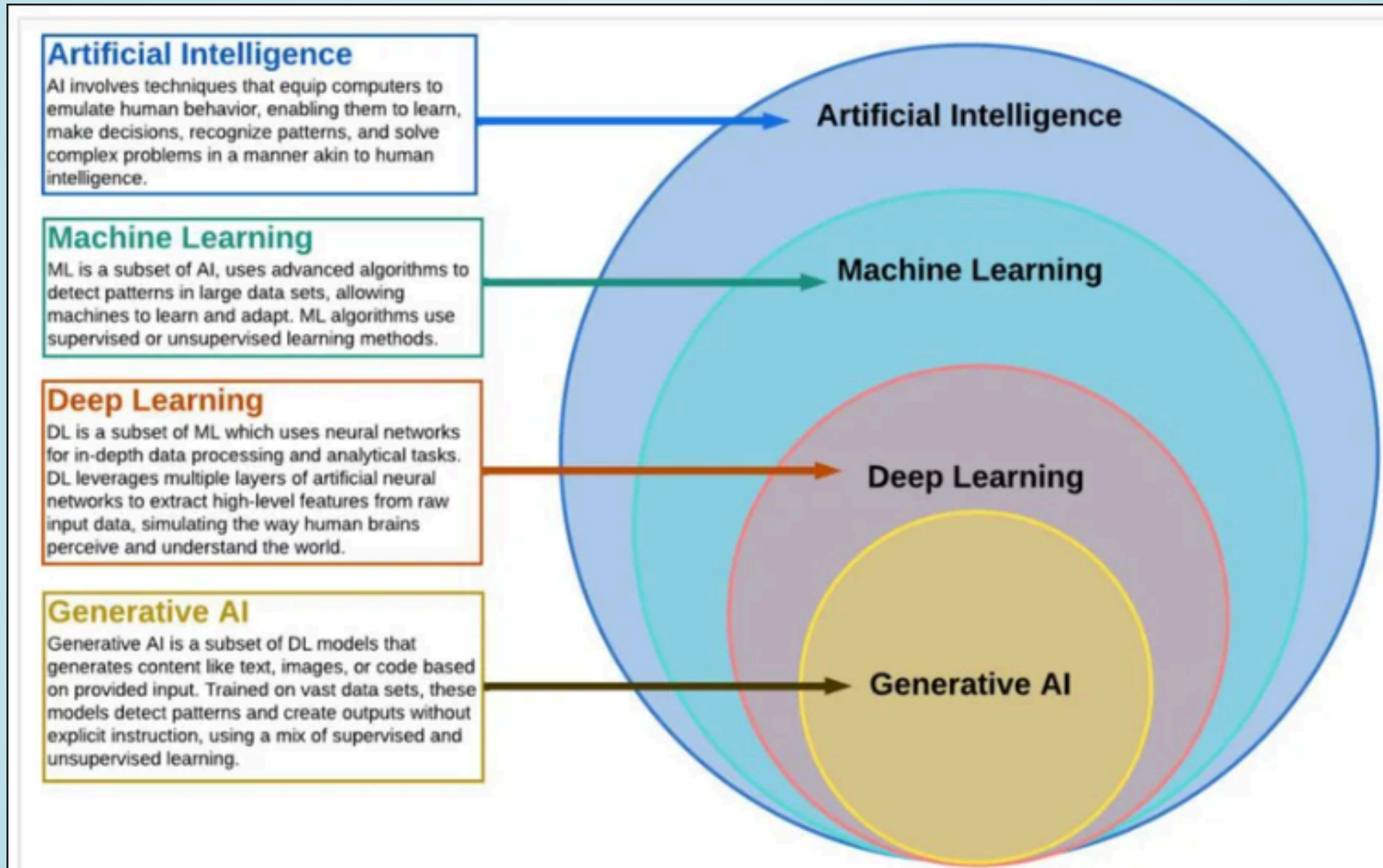
Neural Networks date back decades, so why the resurgence?



Why Deep Learning Now?

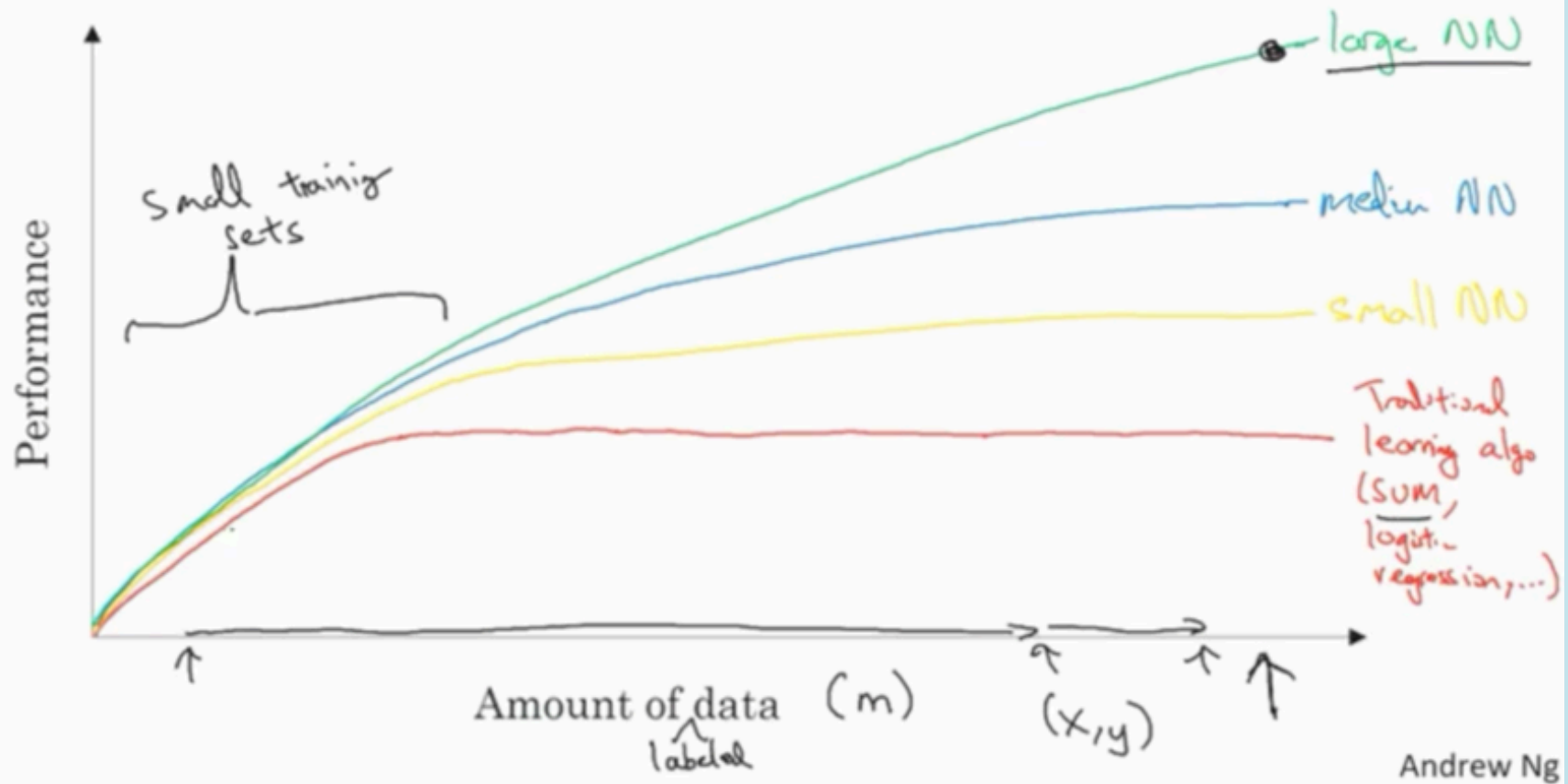
Source: Alex Amini's MIT Introduction to Deep Learning' course

HOW DOES IT ALL FIT?



SIZE DOES MATTER!

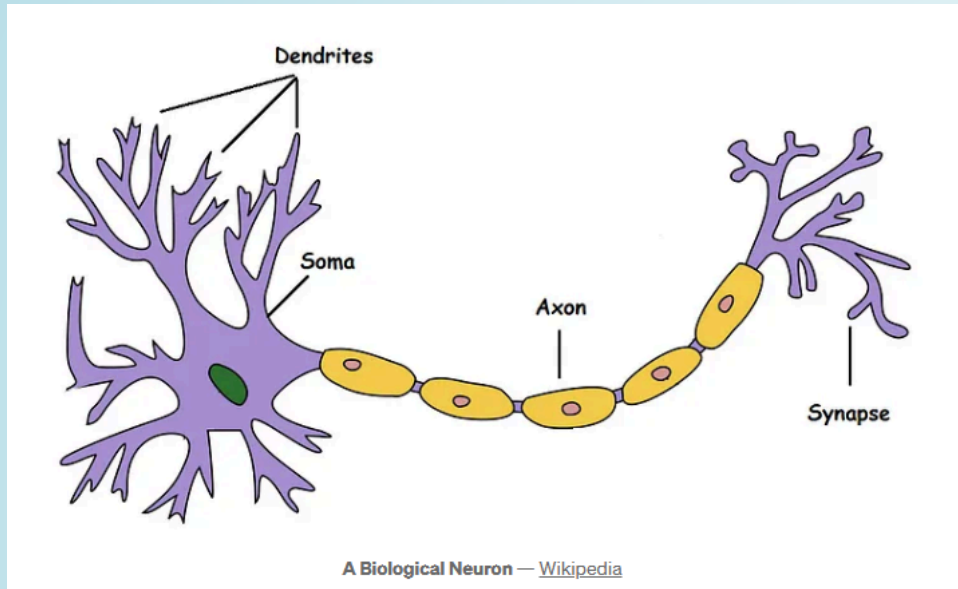
Scale drives deep learning progress



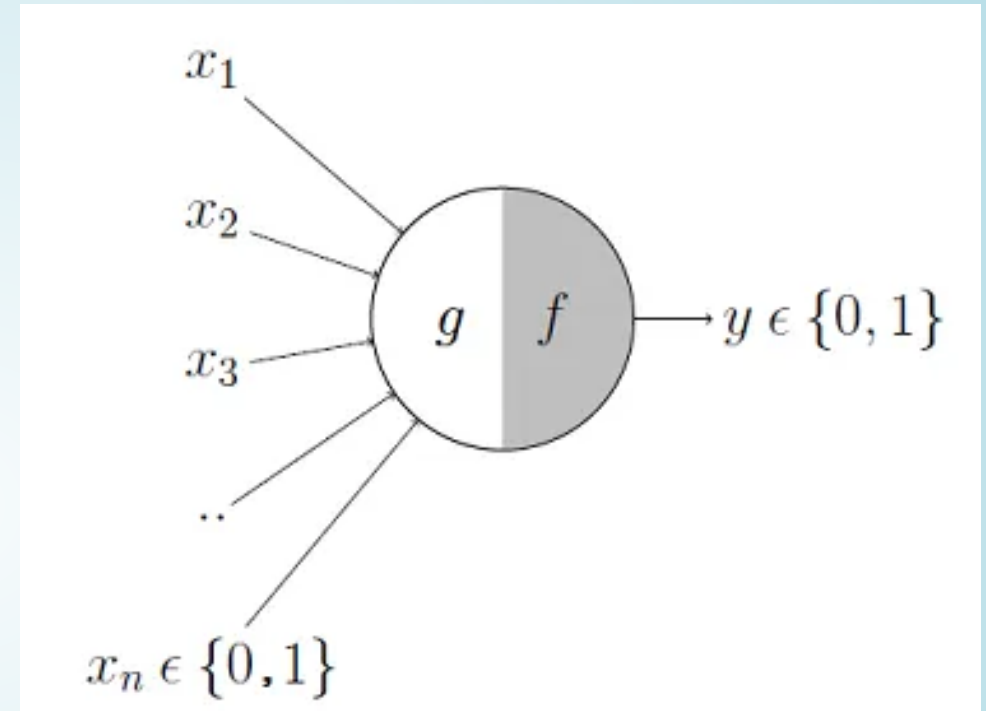
Performance comparison between Deep Learning and other ML algorithms
DL modeling from large amounts of data can increase the performance

THE ARTIFICIAL NEURONE

EMULATING BIOLOGICAL NEURONS



A biological Neuron



MuCulloch & Pitts proposal

- First model of an artificial neurone proposed by Mc Cullough & Pitts in 1943
- They aimed at building a mathematical model of a neurone.

MC CULLOUGH'S NEURON

- The neuron can be divided into two parts:
 - g : combines the input signals,
 - f : yields final output from the aggregated value.
- The neuron computes:

$$y = f(g(x_1, \dots, x_n)).$$

- The neuron first aggregates, $g()$, the inputs, (x) , and then
- Applies decision rule $f()$ to produce a binary output $y \in \{0, 1\}$.

FROM THE AN TO THE PERCEPTRON

Mc Cullogh's neuron has important limitations:

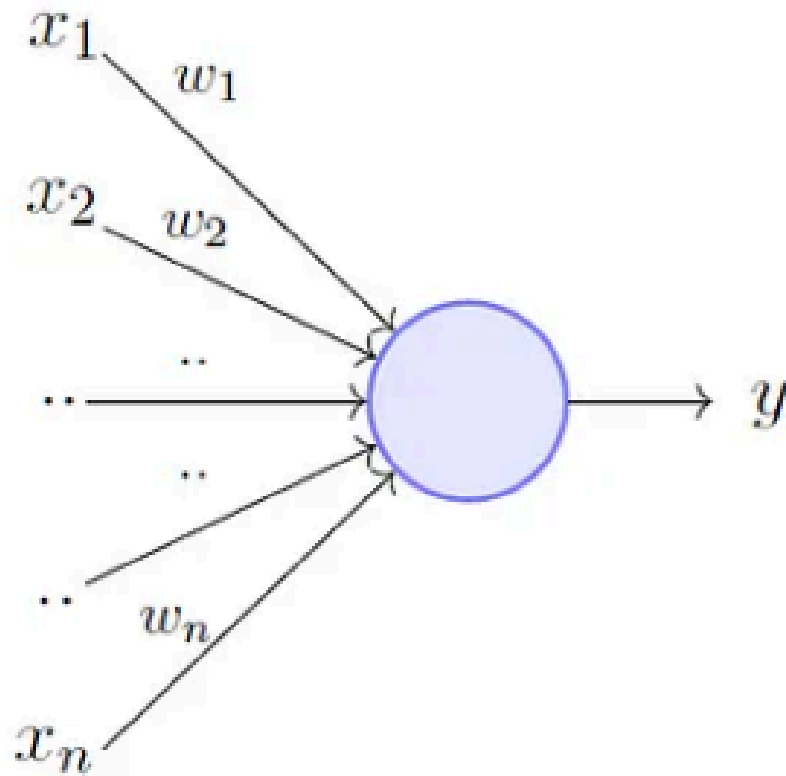
- Inputs are binary: $x_i \in \{0, 1\}$
- All inputs are treated in a fixed way
- The decision rule is rigid (fixed threshold)

To build more flexible the perceptron was introduced.

THE PERCEPTRON

- To overcome Mc Cullogh's neuron limitations Rosenblatt, proposed the perceptron model, or *artificial neuron*, in 1958.
- It generalizes previous one in that *weights and thresholds can be learnt over time*.
 - It takes a weighted sum of the inputs and
 - It sets the output to 1 if the weighted sum exceeds a threshold θ , and to 0 otherwise.

ROSENBLATT'S PERCEPTRON



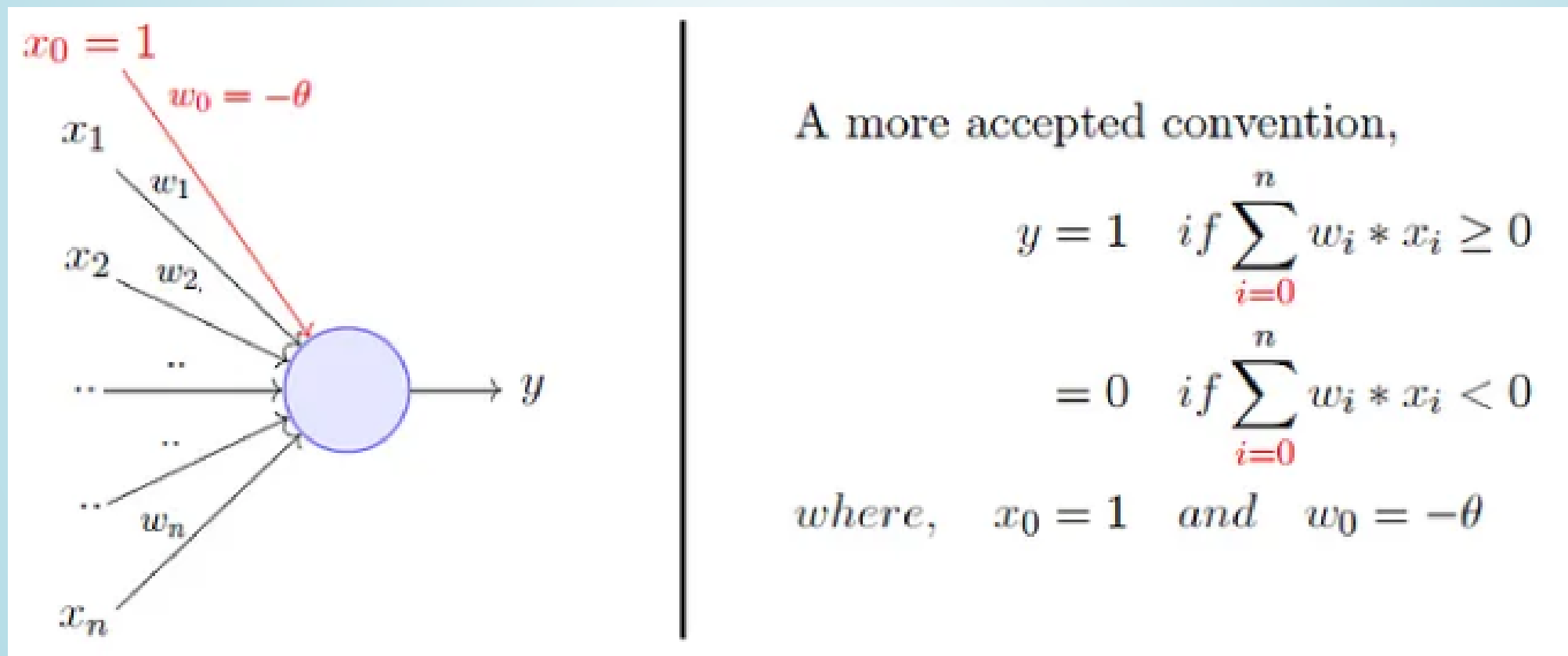
$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i \geq \theta$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta < 0$$

ROSENBLATT'S PERCEPTRON

- Instead of hand coding the thresholding parameter θ ,
- It is added as one of the inputs, with the weight $w_0 = -\theta$.



LIMITATIONS OF THE PERCEPTRON

The perceptron represents an improvement:

- It admits real-valued inputs.
- Both weights and the bias can be learned.

However, it still has important limitations:

- It defines a **linear decision boundary**.
- Therefore, it can only classify **linearly separable data**.

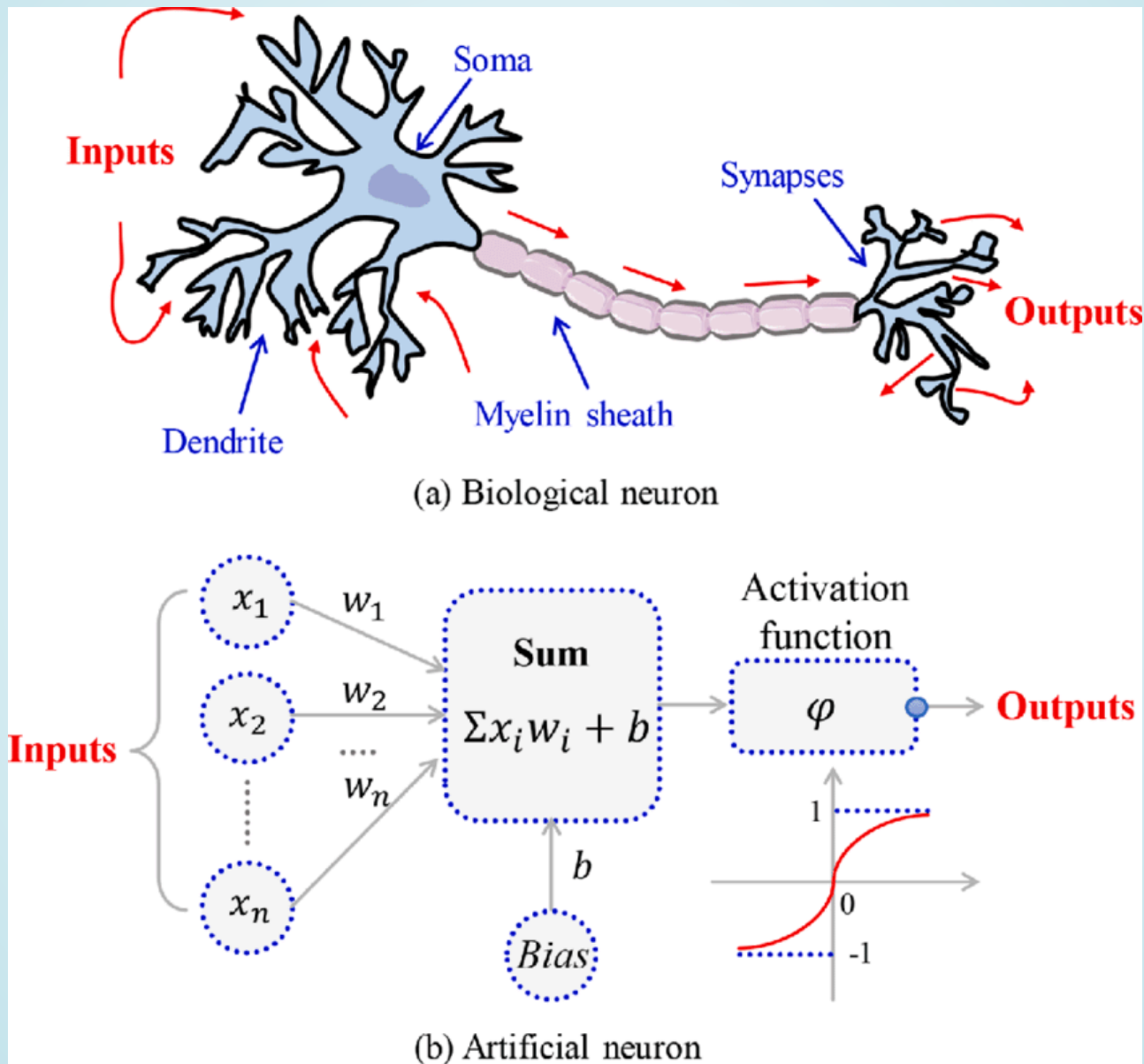
ANS AND ACTIVATION FUNCTIONS

- The Perceptron can be generalized by *Artificial Neurones* which use functions, called *Activation Functions (AFs)* to produce their output.
- AFs are built in a way that they allow neurons to produce *continuous* and *non-linear outputs*.
- It must be noted however that a single AN, even with a different AF, still cannot model non-linear separable problems: *Non-linearity is in the output, no in the input*

BIOLOGICAL VS ARTIFICIAL NEURONS

- Biological neurons are specialized cells that transmit signals to communicate with each other.
- Neuron's activation is based on releasing *neurotransmitters*, chemicals that flow between nerve cells.
 - When the signal reaching the neuron exceeds a certain threshold, it releases neurotransmitters to continue the communication process.
- Artificial Neurons rely on Activation Functions to emulate the activation process of biological neurons

ACTIVATION FUNCTION



ARTIFICIAL NEURON

With all these ideas in mind we can now define an Artificial Neuron as a *computational unit* that :

- takes as input
 $x = (x_0, x_1, x_2, x_3), (x_0 = +1 \equiv \text{bias}),$
- outputs $h_{\theta}(x) = f(\theta^{\top} x) = f(\sum_i \theta_i x_i),$
- where $f : \mathbb{R} \mapsto \mathbb{R}$ is called the **activation function**.

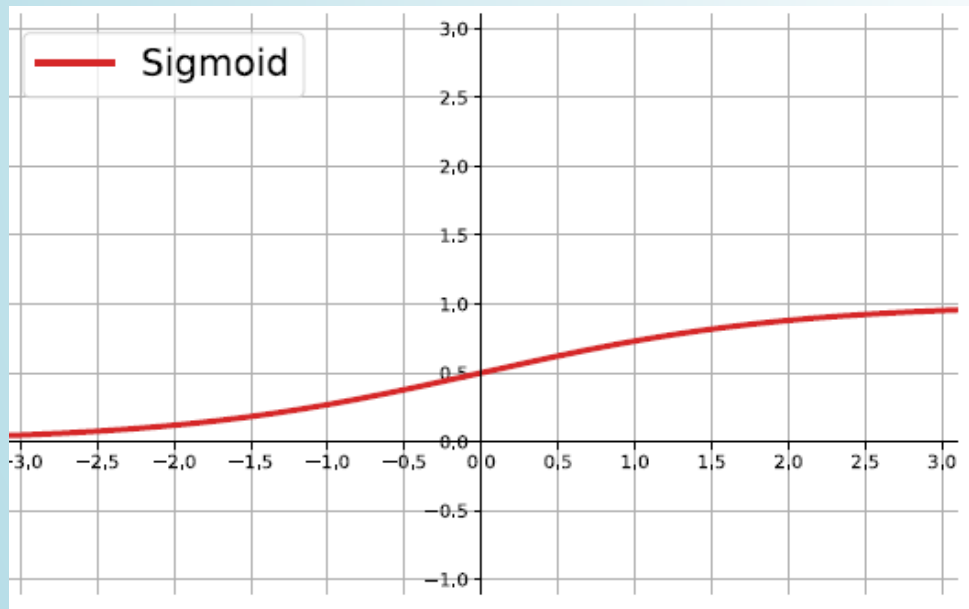
ACTIVATION FUNCTIONS

- Goal of activation function is to provide the neuron with *the capability of producing the required outputs*.
- Flexible enough to produce
 - Either linear or non-linear transformations.
 - Output in the desired range ($[0,1]$, $\{-1,1\}$, $\mathbb{R}^+$...)
- Usually chosen from a (small) set of possibilities.
 - Sigmoid function
 - Hyperbolic tangent, or **tanh**, function
 - ReLU

THE SIGMOID FUNCTION

$$f(z) = \frac{1}{1 + e^{-z}}$$

- Output real values $\in (0, 1)$.
- Natural interpretations as *probability*



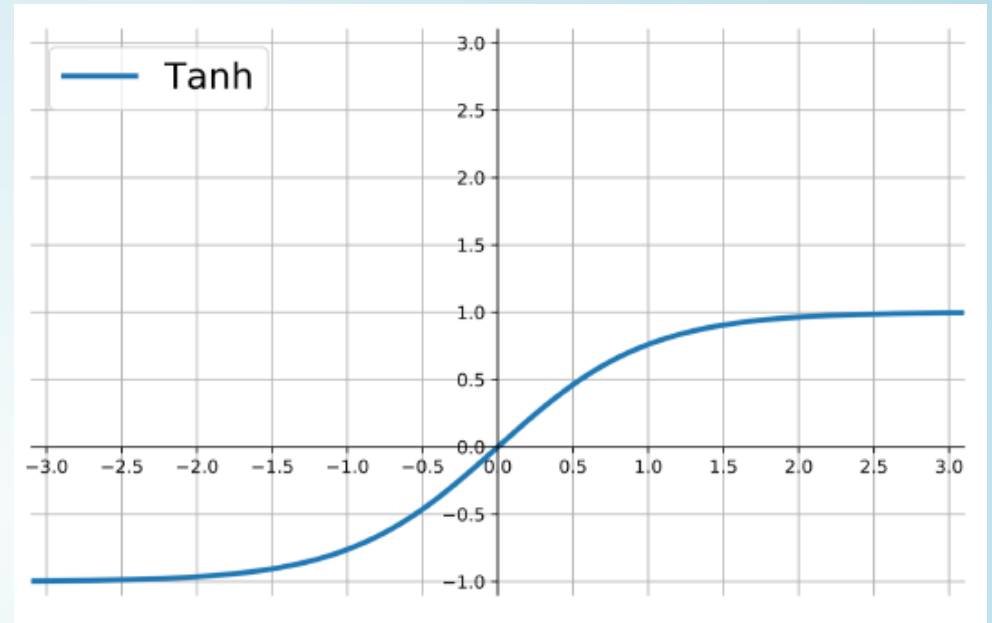
$$\begin{aligned}\frac{d}{dx} \sigma(x) &= \frac{d}{dx} \left[\frac{1}{1 + e^{-x}} \right] & \frac{d}{dx} e^x &= e^x \\ &= \frac{d}{dx} (1 + e^{-x})^{-1} \\ &= -(1 + e^{-x})^{-2} (-e^{-x}) \\ &= \frac{e^{-x}}{(1 + e^{-x})^2} \\ &= \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} \\ &= \frac{1}{1 + e^{-x}} \cdot \frac{(1 + e^{-x}) - 1}{1 + e^{-x}} \\ &= \frac{1}{1 + e^{-x}} \cdot \left(\frac{1 + e^{-x}}{1 + e^{-x}} - \frac{1}{1 + e^{-x}} \right) \\ &= \frac{1}{1 + e^{-x}} \cdot \left(1 - \frac{1}{1 + e^{-x}} \right) \\ &= \sigma(x) \cdot (1 - \sigma(x))\end{aligned}$$

THE HYPERBOLIC TANGENT

Also called **tanh**, function:

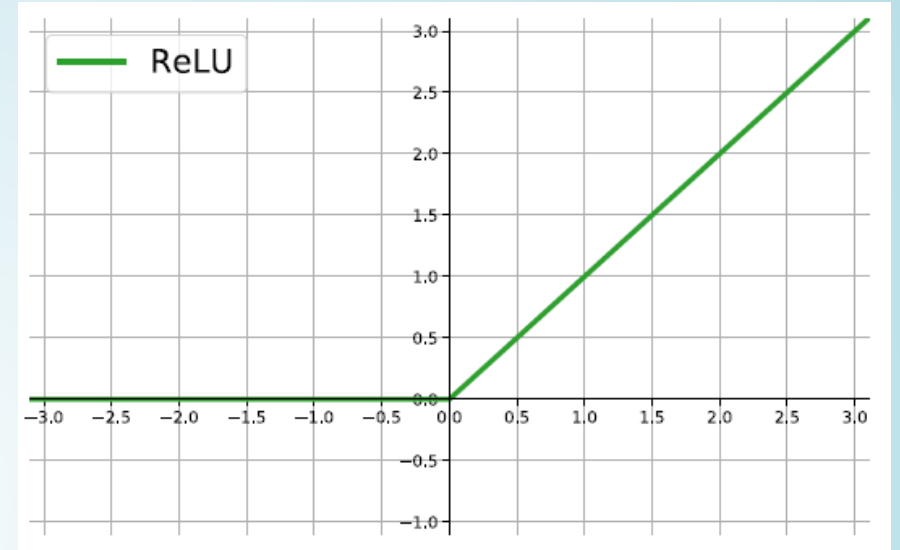
$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- outputs are zero-centered and bounded in $-1,1$
- scaled and shifted Sigmoid
- stronger gradient but still has vanishing gradient problem
- Its derivative is $f'(z) = 1 - (f(z))^2$.



THE RELU

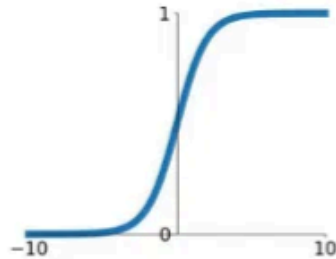
- *rectified linear unit*: $f(z) = \max\{0, z\}$.
- Close to a linear: piece-wise *linear* function with two linear pieces.
- Outputs are in $(0, \infty)$, thus not bounded
- Half rectified: activation threshold at 0
- No vanishing gradient problem



MORE ACTIVATION FUNCTIONS

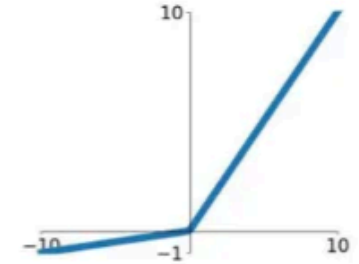
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



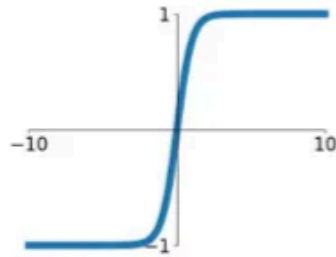
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

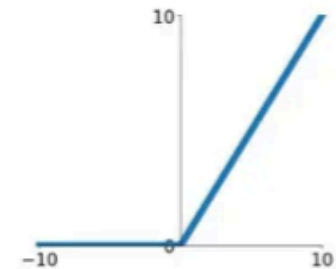


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

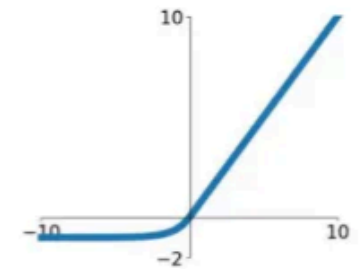
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



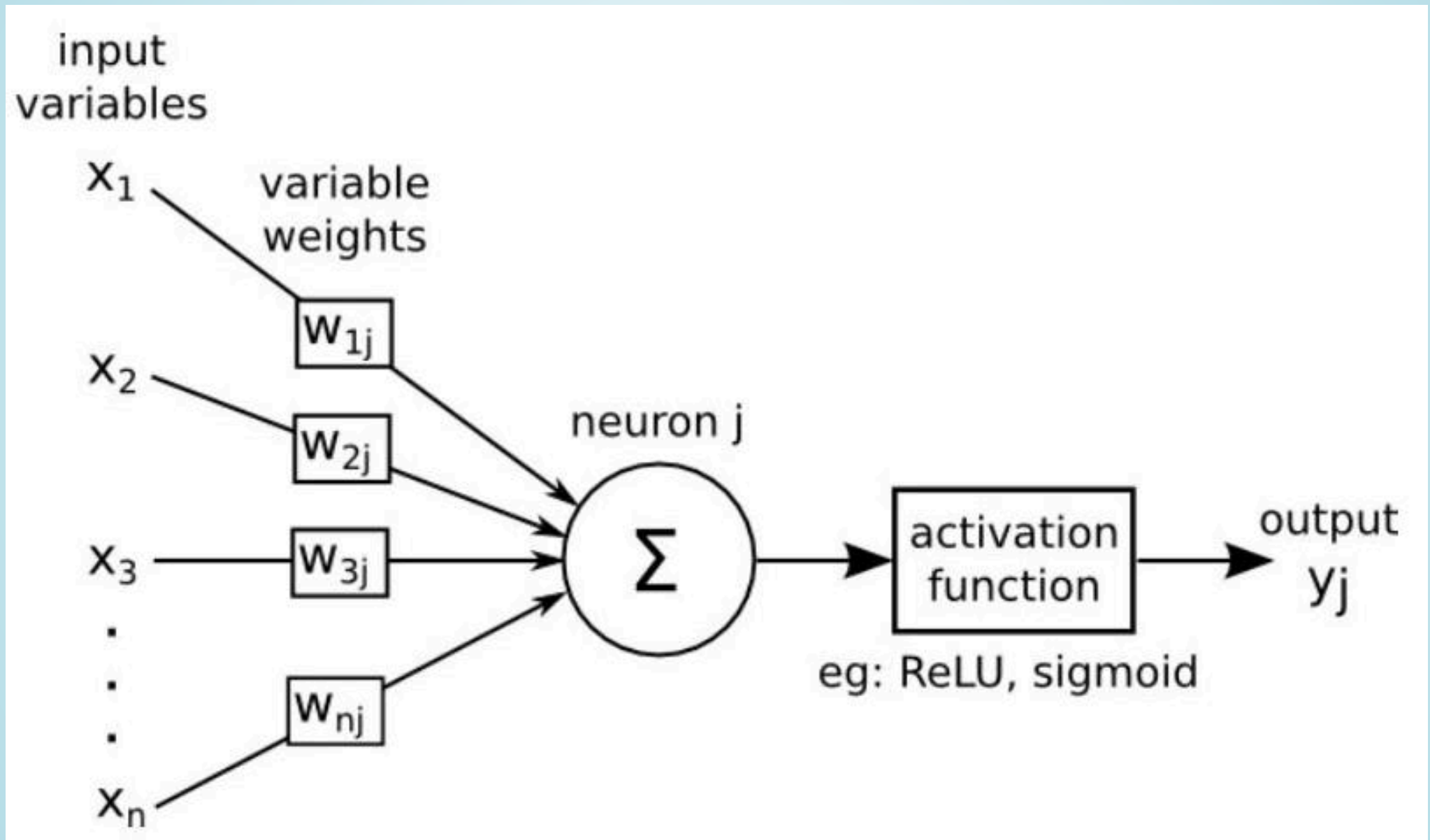
SOFTMAX ACTIVATION FUNCTION

- **Softmax** is an activation function used in the **output layer** of classification models, especially for **multi-class problems**.
- It converts raw scores (logits) into **probabilities**, ensuring that $\sum_{i=1}^N P(y_i) = 1$ where $P(y_i)$ is the predicted probability for class i .
- Given an input vector z , *Softmax* transforms it as:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

- The exponentiation **amplifies differences**, making the highest value more dominant.
- The normalization **ensures that probabilities sum to 1**.

THE ARTIFICIAL NEURON IN SHORT



THE ARTIFICIAL NEURON IN SHORT

- An AN takes a vector of input values $x_1, \dots, x_d$ and combines it with some weights that are local to the neuron ($w_0, w_1, \dots, w_d$) to compute a net input $w_0 + \sum_{i=1}^d w_i \cdot x_i$.
- To compute its output, it then passes the net input through a possibly non-linear univariate activation function $g(\cdot)$, usually chosen from a set of options such as *Sigmoid*, *Tanh* or *ReLU* functions
- To deal with the *bias*, we create an extra input variable x_0 with value always equal to 1, and so the function computed by a single artificial neuron (parameterized by its weights $\mathbf{w}$) is:

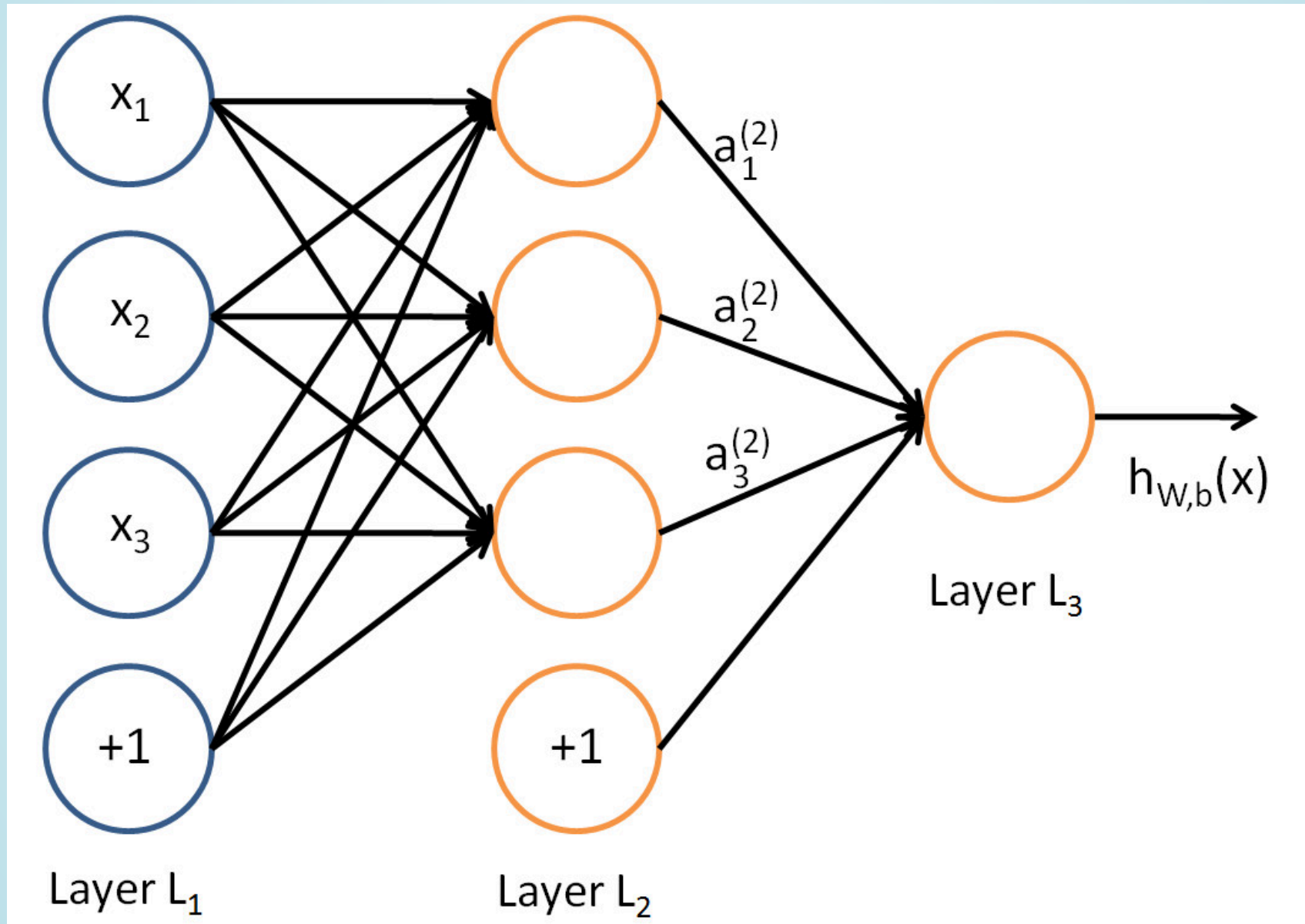
$$y(\mathbf{x}) = g \left(w_0 + \sum_{i=1}^d w_i x_i \right) = g \left(\sum_{i=0}^d w_i x_i \right) = g (\mathbf{w}^T \mathbf{x})$$

FROM NEURONS TO NEURAL NETWORKS

THE BASIC NEURAL NETWORK

- Continuing with the brain analogy one can combine (artificial) neurons to create better learners.
- A simple artificial neural network is created by two types of modifications to the basic Artificial Neuron.
 - *Stacking several neurons* insteads of just one.
 - Adding an additional layer of neurons, which is call a *hidden layer*,
- This yields a system where *the output of a neuron can be the input of another* in many different ways.

AN ARTIFICIAL NEURAL NETWORK



THE ARCHITECTURE OF ANN

In this figure, we have used circles to also denote the inputs to the network.

- Circles labeled +1 are *bias units*, and correspond to the intercept term.
- The leftmost layer of the network is called the *input layer*.
- The rightmost layer of the network is called the *output layer*.
- The middle layer of nodes is called the *hidden layer*, because its values are not observed in the training set.

Bias nodes are not counted when stating the neuron size.

With all this in mind our example neural network has three layers with:

- 3 input units (not counting the bias unit),
- 3 hidden units,
- 1 output unit.

NEURAL NETWORKS AS FUNCTIONS

A neural network defines a parametric function:

$$\hat{y} = f(x; \theta)$$

where: - x is the input, - $\hat{y}$ is the output, - $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^L$ are the parameters.

The function f is obtained by *composing linear and non-linear transformations*:

$$f(x) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(x)$$

HOW AN ANN WORKS

An ANN is a predictive model (a *learner*) whose properties and behaviour can be well characterized.

- It operates through a process known as *forward propagation*, which encompasses the information flow from the input layer to the output layer.
- Forward propagation is performed by composing a series of linear and non-linear (activation) functions.
- These are characterized (parametrized) by their *weights* and *biases*, that need to be *learnt*.
 - This is done by *training the ANN*.

FORWARD PROPAGATION (1)

The process that encompasses the computations required to go from the input values to the final output is known as *forward propagation*.

For the ANN with 3 input values and 3 neurons in the hidden layer we have:

1. Each node $a_i^{(2)}$ of the hidden layer operates on all input values:

$$a_1^{(2)} = f(\theta_{10}^{(1)} + \theta_{11}^{(1)}x_1 + \theta_{12}^{(1)}x_2 + \theta_{13}^{(1)}x_3)$$

$$a_2^{(2)} = f(\theta_{20}^{(1)} + \theta_{21}^{(1)}x_1 + \theta_{22}^{(1)}x_2 + \theta_{23}^{(1)}x_3)$$

$$a_3^{(2)} = f(\theta_{30}^{(1)} + \theta_{31}^{(1)}x_1 + \theta_{32}^{(1)}x_2 + \theta_{33}^{(1)}x_3)$$

2. Output layer:

$$h_{\Theta}(x) = a_1^{(3)} = f(\theta_{10}^{(2)} + \theta_{11}^{(2)}a_1^{(2)} + \theta_{12}^{(2)}a_2^{(2)} + \theta_{13}^{(2)}a_3^{(2)})$$

The terms $\theta_{i0}^{(l)}$ act as bias parameters.

FORWARD PROPAGATION (2)

Forward propagation can be written compactly as:

$$z^{(l+1)} = W^{(l)} a^{(l)} + b^{(l)}$$

$$a^{(l+1)} = f(z^{(l+1)})$$

where:

- $W^{(l)}$ contains the weights,
- $b^{(l)}$ contains the bias terms,
- $f(\cdot)$ is applied elementwise.

This form is used in most implementations.

FORWARD PROPAGATION (3)

An alternative compact notation incorporates the bias into the weight matrix.

Standard form (explicit bias)

$$z^{(l+1)} = W^{(l)} a^{(l)} + b^{(l)}$$

$$a^{(l+1)} = f(z^{(l+1)})$$

- $W^{(l)}$: weights
- $b^{(l)}$: bias
- $f(\cdot)$: elementwise

Most common in implementations

Augmented form (bias included)

$$\tilde{a}^{(l)} = \begin{bmatrix} 1 \\ a^{(l)} \end{bmatrix}$$

$$z^{(l+1)} = \Theta^{(l)} \tilde{a}^{(l)}$$

$$a^{(l+1)} = f(z^{(l+1)})$$

- $\Theta^{(l)} = [b^{(l)} \quad W^{(l)}]$

Same model, more compact notation

ANNS AS COMPOSITIONS

In short, a neural network defines a parametric function:

$$\hat{y} = f(x; \theta)$$

where $f(\cdot)$ is obtained by *composing a sequence of transformations*:

$$f(x) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(x)$$

Each layer defines a transformation of the form:

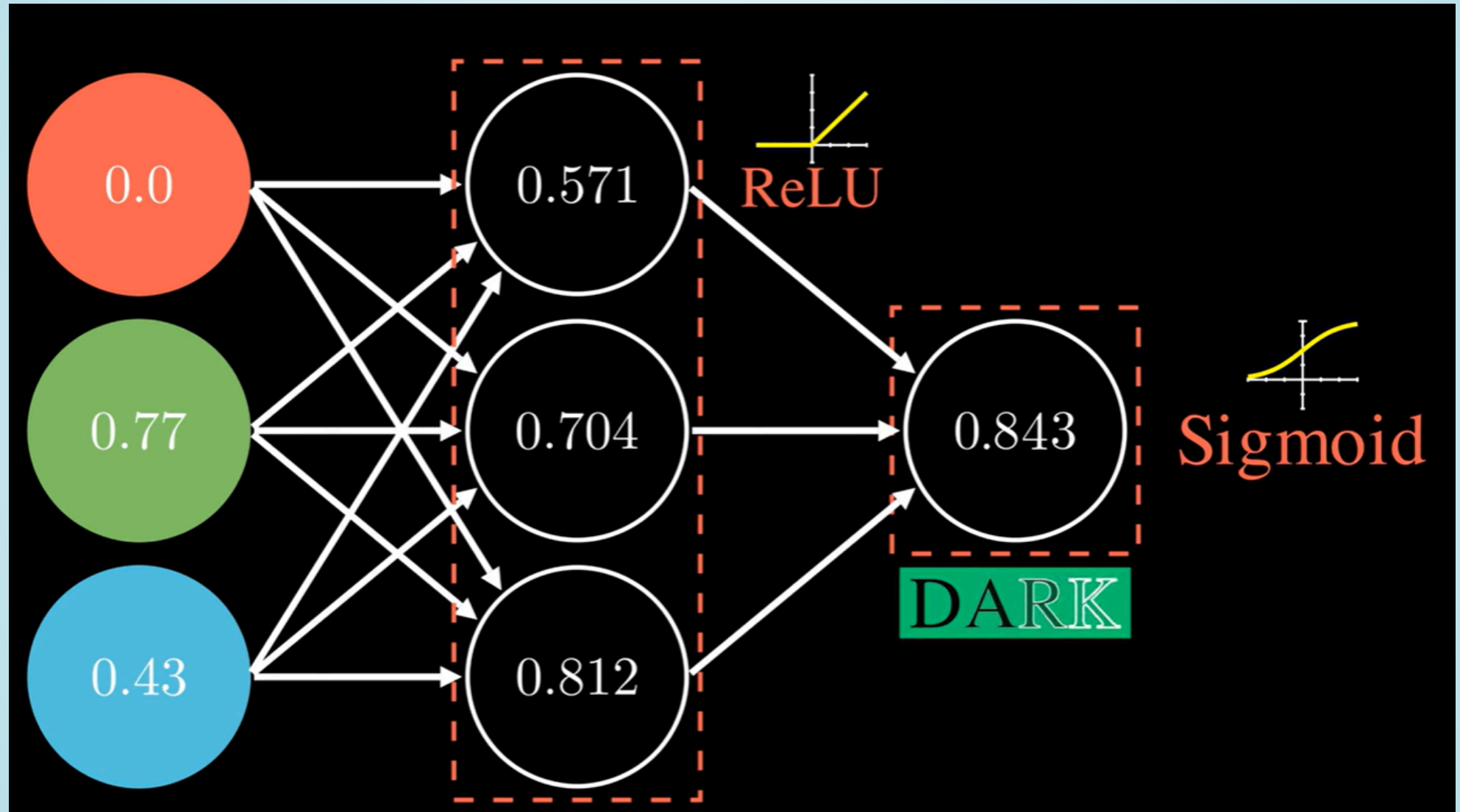
$$f^{(l)}(a) = f(W^{(l)}a + b^{(l)})$$

so that the activations propagate as:

$$a^{(l+1)} = f^{(l)}(a^{(l)})$$

with: $a^{(1)} = x$ (input layer) and $a^{(L)} = \hat{y}$ (output layer).

A SIMPLE NEURAL NETWORK



Example from youtube: A simple Neural network in 3 minutes

FEED FORWARD NNS

- The type of NN described is called feed-forward *neural network* (FFNN), since
 - All computations are done by Forward propagation
 - The connectivity graph does not have any directed loops or cycles.

EFICIENT FORWARD PROPAGATION

- The way input data is transformed, through a series of weightings and transformations, until the ouput layer is called *forward propagation*.
- By organizing parameters in matrices, and using matrix-vector operations, fast linear algebra routines can be used to perform the required calculations in a fast efficient way.

MULTIPLE ARCHITECTURES FOR ANN

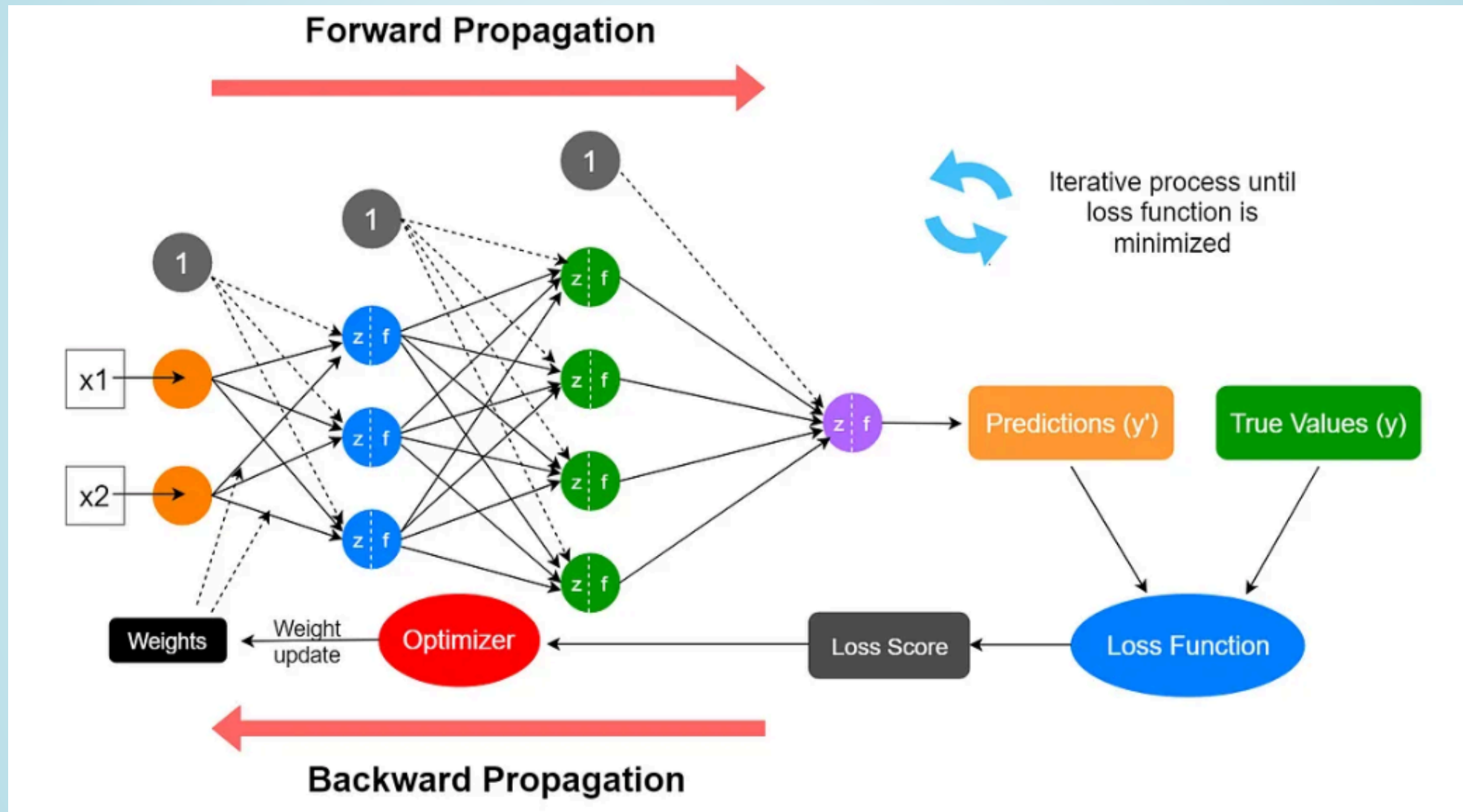
- We have so far focused on a single hidden layer neural network of the example.
- One can, however build neural networks with many distinct architectures (meaning patterns of connectivity between neurons), including ones with multiple hidden layers.
- See [here the Neural Network Zoo](#).

TRAINING NEURAL NETWORKS

TRAINING AN ANN

- An ANN is a predictive model whose properties and behaviour can be mathematically characterized.
- In practice this means:
 - The ANN acts by composing a series of linear and non-linear (activation) functions.
 - These are characterized by their *weights* and *biases*, that need to be *learnt*.
- *Training* the network is done by
 - Selecting an appropriate (convex) loss function,
 - Finding weights that minimize a the total cost function (avg loss).

TRAINING AN ANN



Source: Overview of a Neural Network's Learning Process

THE TOOLS FOR TRAINING

- Training an ANN is usually done using some iterative optimization procedure such as *Gradient Descent*.
- This requires evaluating derivatives in a huge number of points.
 - Such high number may be reduced by *Stochastic Gradient Descent*.
 - The evaluation of derivatives is simplified thanks to *Backpropagation*.

LOSS FUNCTIONS FOR OPTIMIZATION

Depending on the activation function it may be advisable to use one or another form of loss function.

- A typical choice may be *quadratic (or square) error loss*:

$$l(h_{\theta}(x), y) = \left(y - \frac{1}{1 + e^{-\theta^T x}} \right)^2$$

- Given a sigmoid AF, the squared error loss *is not a convex problem* so that MSE is not appropriate.
- Quadratic loss may be used with ReLu activation.

CROSS-ENTROPY LOSS FUNCTION

- A common loss function to use with ANNs is Cross-entropy defined as:

$$l(h_{\theta}(x), y) = \begin{cases} -\log h_{\theta}(x) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

- This function can also be written as:

$$l(h_{\theta}(x), y) = -y \log h_{\theta}(x) - (1 - y) \log(1 - h_{\theta}(x))$$

- Using cross-entropy loss, the cost function is of the form:

$$J(\theta) = -\frac{1}{n} \left[\sum_{i=1}^n (y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))) \right]$$

- Now, this is a convex optimization problem.

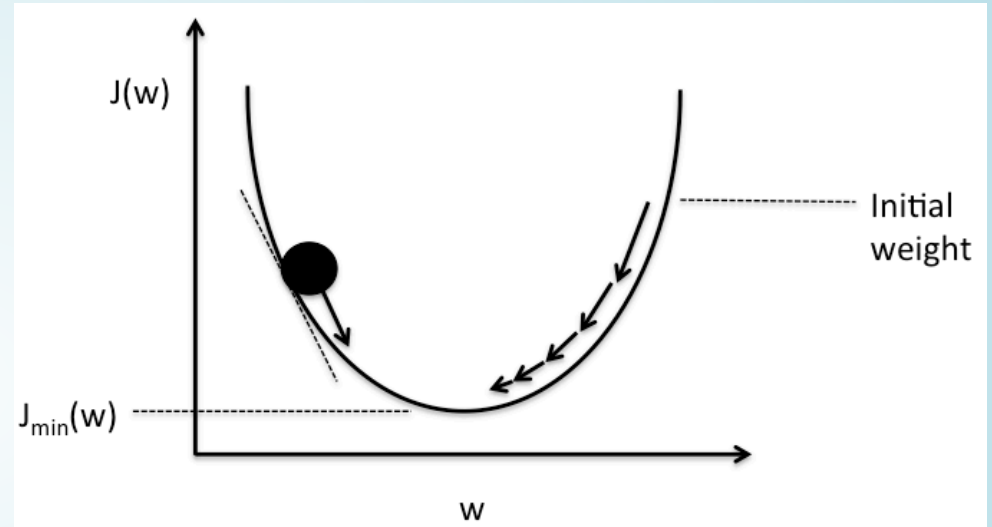
THE PARAMETERS FOR TUNING

- Training a network corresponds to finding the parameters, that is, *the weights* and *the biases*, that minimize the cost function.
- Although weights & biases are respectively matrices and vectors, it is convenient to represent them in a vectorized form stored as a single vector, that will be denoted here, by θ .
- We may suppose $\theta \in \mathbb{R}^p$, and write the cost function as $J(\theta)$ to emphasize its dependence on the parameters, that is:

$$\begin{aligned} J : \mathbb{R}^p &\rightarrow \mathbb{R} \\ \theta &\rightarrow J(\theta) \end{aligned}$$

GRADIENT OR *STEEPEST* DESCENT

- A classical method in optimization to minimize a convex function $J(\theta)$.
- It proceeds iteratively, computing a sequence of vectors $\theta^1, \theta^2, \dots, \theta^n$ in $\mathbb{R}^p$ with the aim of converging to a vector that minimizes the cost function.



GRADIENT DESCENT

- Suppose that our current vector is θ . How should we choose a perturbation, $\Delta\theta$, so that the next vector, $\theta + \Delta\theta$, represents an improvement, that is: $J(\theta + \Delta\theta) < J(\theta)$?
- Linearize the cost function using a Taylor approximation.
- If $\Delta\theta$ is small, then ignoring terms of order $\|\Delta\theta\|^2$ or higher:

$$J(\theta + \Delta\theta) \approx J(\theta) + \sum_{i=1}^p \frac{\partial J(\theta)}{\partial \theta_i} \Delta\theta_i$$

or, equivalently:

$$J(\theta + \Delta\theta) \approx J(\theta) + \nabla J(\theta)^\top \Delta\theta$$

where $\nabla J(\theta) \in \mathbb{R}^p$ denote the *gradient*, i.e. the vector of partial derivatives:

$$\nabla J(\theta) = \left(\frac{\partial J(\theta)}{\partial \theta_1}, \dots, \frac{\partial J(\theta)}{\partial \theta_p} \right)^\top$$

GRADIENT DESCENT

- Goal: choose a perturbation, $\Delta\theta$, s.t.: $J(\theta + \Delta\theta) < J(\theta)$
- Taylor approximation above suggests that choosing $\Delta\theta$ to *make* $\nabla J(\theta)^\top \Delta\theta$ *negative* will make the value of $J(\theta + \Delta\theta)$ smaller.
- Indeed it can be shown that the highest possible negative value will come out when $-\nabla J(\theta) = \Delta\theta$, which leads to the gradient descent formula:

$$\theta \rightarrow \theta - \eta \nabla J(\theta),$$

where η , the *learning rate* is the size of the step taken at each iteration, which should be small because of Taylor App.

THE CAUCHY-SCHWARZ INEQUALITY AND GRADIENT DESCENT

- The [Cauchy-Schwarz inequality](#), states that for any $f, g \in \mathbb{R}^p$, we have:

$$|f^\top g| \leq ||f|| \cdot ||g||.$$

- Moreover, the two sides are equal if and only if f and g are linearly dependent (meaning they are parallel).
- By Cauchy-Schwarz, biggest possible value for $\nabla J(\theta)^\top \Delta\theta$ is the upper bound, $||\nabla J(\theta)|| \cdot ||\Delta\theta||$.
 - The equality is only reached when $||\nabla J(\theta)|| = ||\Delta\theta||$
- This explains why we choose precisely $-\nabla J(\theta) = \Delta\theta$

GRADIENT DESCENT

In summary, given a cost function $J(\theta)$ to be optimized the gradient descent optimization proceeds as follows:

1. **Initialize** θ_0 randomly or with some predetermined values
2. **Repeat until convergence:**

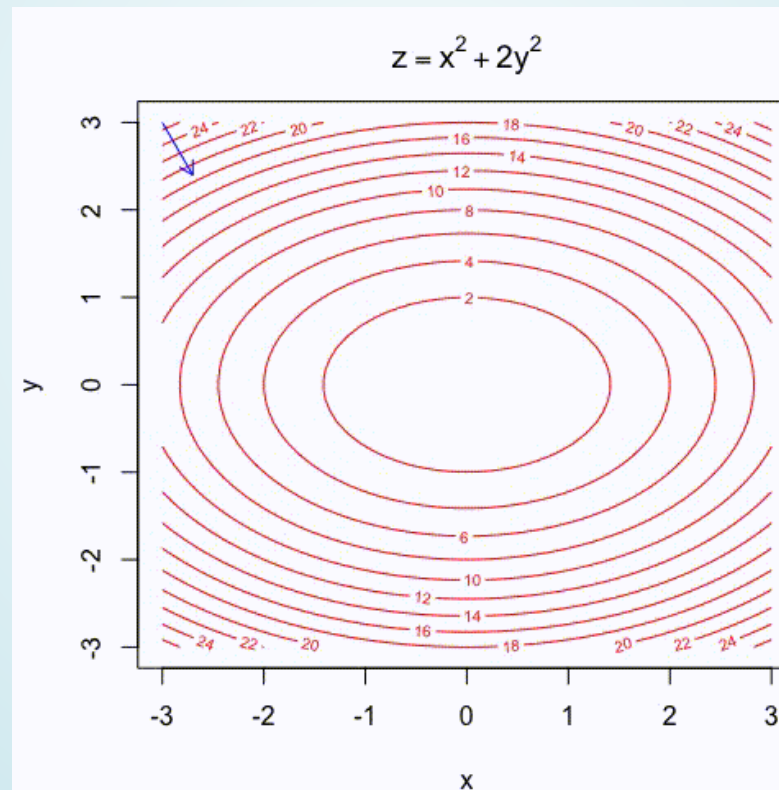
$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

3. **Stop when:** $|J(\theta_{t+1}) - J(\theta_t)| < \epsilon$

- θ_0 is the initial parameter vector,
- θ_t is the parameter vector at iteration t ,
- η is the learning rate,
- $\nabla J(\theta_t)$ is the gradient of the loss function with respect to θ at iteration t ,
- ϵ is a small positive value indicating the desired level of convergence.

GRADIENT DESCENT ILLUSTRATION

- Gradient descent is an intuitive approach that has been thoroughly illustrated in many different ways:



<https://assets.yihui.org/figures/animation/example/grad-desc>

COMPUTING GRADIENTS

- The gradient method optimizes weights and biases $\theta = \{W, b\}$ by minimizing the cost function $J(\theta)$.
- This requires computing partial derivatives:

$$\frac{\partial}{\partial \theta_j} J(\theta)$$

- The algorithm used for this computation is called backpropagation.

A SHORT HISTORY

- Introduced in the 1970s in an MSc thesis.
- In 1986, [Rumelhart, Hinton, and Williams](#) demonstrated that backpropagation significantly improves learning speed.
- This breakthrough enabled neural networks to solve previously intractable problems.

BACKPROPAGATION INTUITION

- The term originates from error backpropagation.
- An artificial neural network computes outputs through forward propagation:
 - Input values pass through linear and non-linear transformations.
 - The network produces a prediction.
- The error (difference between predicted and true value) is:
 - Propagated backward to compute the error contribution of each neuron.
 - Used to adjust weights iteratively.

DELTA RULE AND ERROR TERM δ

- The delta rule updates weights based on the error term δ_j , which measures how much neuron j contributes to the error.
- It is computed as:

$$\delta_j = \sigma'(z_j)(c_j - y_j)$$

where:

- $\sigma'(z_j)$ is the derivative of the activation function.
- $(c_j - y_j)$ is the error in neuron j .

WEIGHT UPDATE ΔW

- Once δ_j is computed, weights are updated using:

$$\Delta w_i^j = \eta \delta_j x_i^j$$

where:

- η is the learning rate.
 - δ_j determines the direction and magnitude of the update.
 - x_i^j is the input to neuron j .
- In hidden layers, δ is propagated backward:

$$\delta_k = \sigma'(z_k) \sum_{j \in S_k} \delta_j w_k^j$$

where S_k is the set of output neurons connected to neuron k .

BACKPROPAGATION ALGORITHM

1. Forward propagation

- Compute the network output for a given input.

2. Backward propagation

- Compute δ_j for each neuron in the output layer.
- Propagate δ_j backward to compute δ_k for hidden layers.
- Update weights using gradient descent.

FINAL ALGORITHM

Input: W (weight vectors), D (training dataset).

Repeat until error is below threshold:

1. Compute network output for each training instance.
2. For each neuron j in the output layer:
 - Compute $\delta_j = \sigma'(z_j)(c_j - y_j)$
 - Update weights: $\Delta w_i^j = \eta \delta_j x_i^j$
3. For each neuron k in hidden layers:
 - Compute $\delta_k = \sigma'(z_k) \sum_{j \in S_k} \delta_j w_k^j$
 - Update weights : $\Delta w_i^k = \eta \delta_k x_i^k$

Output: Updated weight vectors W .

CHAIN RULE IN BACKPROPAGATION

- In multi-layer networks, the error signal must be **propagated backward** through multiple layers.
- The weight update in hidden layers depends not only on the local error but also on how errors propagate from the output layer.
- This is achieved using the **chain rule of calculus**, which decomposes derivatives into simpler components.
- That is, the **gradient descent update rule** for the weight is:

$$\Delta w_i^j = -\eta \frac{\partial J}{\partial w_i^j} = \eta \delta_j x_i^j$$

where the derivative will be computed applying the chain rule.

APPLYING THE CHAIN RULE

- To compute the gradient of the loss function J with respect to a weight w_i^j connecting neuron i to neuron j , we use the chain rule:

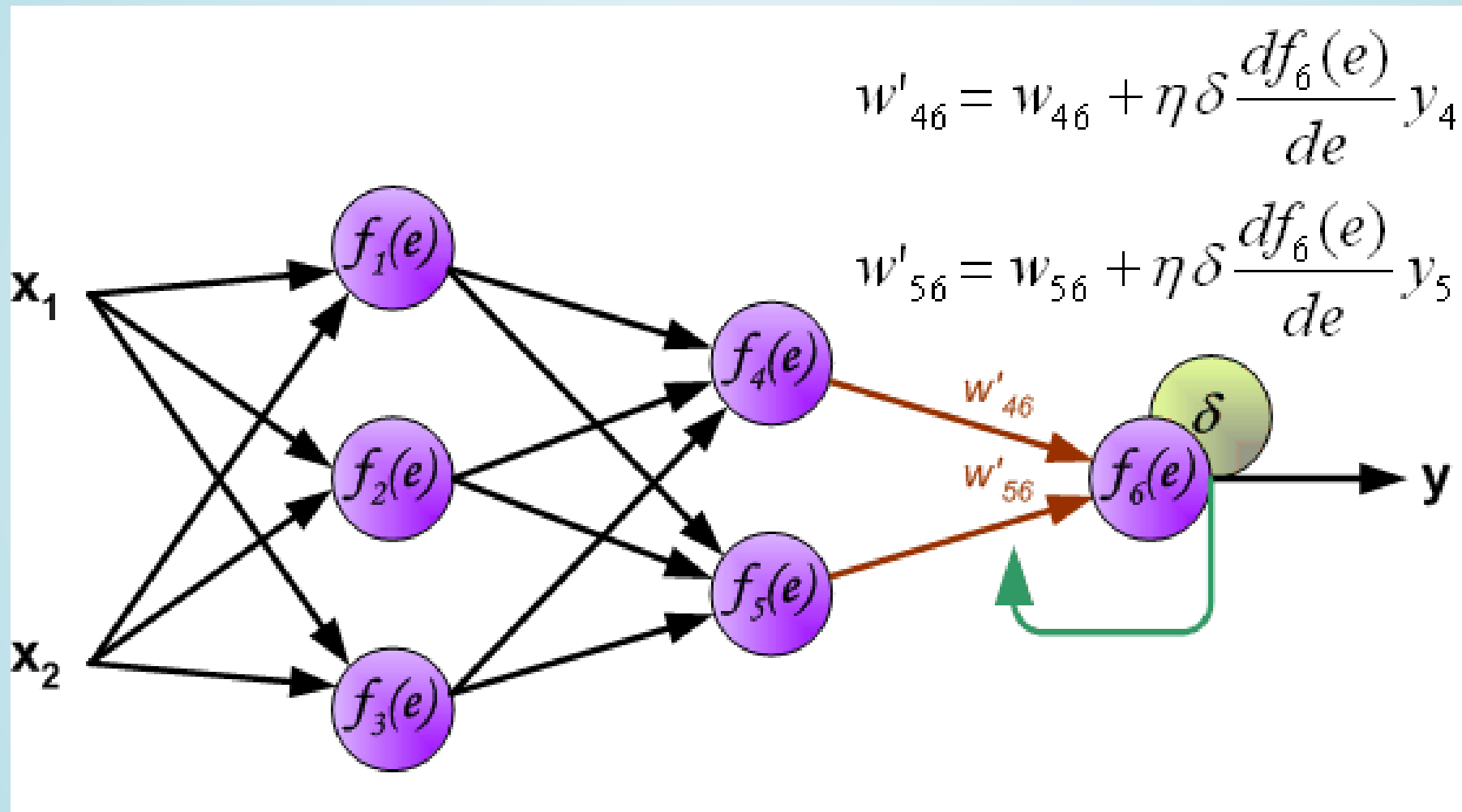
$$\frac{\partial J}{\partial w_i^j} = \frac{\partial J}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_i^j}$$

- $\frac{\partial J}{\partial y_j}$ measures how the loss changes with respect to the output of neuron j .
- $\frac{\partial y_j}{\partial z_j} = \sigma'(z_j)$ is the derivative of the activation function.
- $\frac{\partial z_j}{\partial w_i^j} = x_i^j$ represents the input to the neuron.

AUTOMATIC DIFFERENTIATION

- Modern deep learning frameworks do not compute gradients manually.
- Instead, they use **automatic differentiation** and **computational graphs** to simplify and speed up backpropagation.
- A **computational graph** represents the sequence of operations in a neural network as a directed graph.
 - Each node corresponds to an operation (e.g., addition, multiplication, activation function).
 - This structure allows efficient **backpropagation** by applying the **chain rule automatically**.
- **Automatic differentiation (AD)** relies the computational graph to apply the chain rule and compute gradients automatically in the Backwards pass.
- Frameworks like **TensorFlow, PyTorch, and JAX** use **reverse-mode differentiation**, which is particularly efficient for functions with many parameters (like neural networks).

TUNING A NEURAL NETWORK



IMPROVING THE LEARNING PROCESS

LEARNING OPTIMIZATION

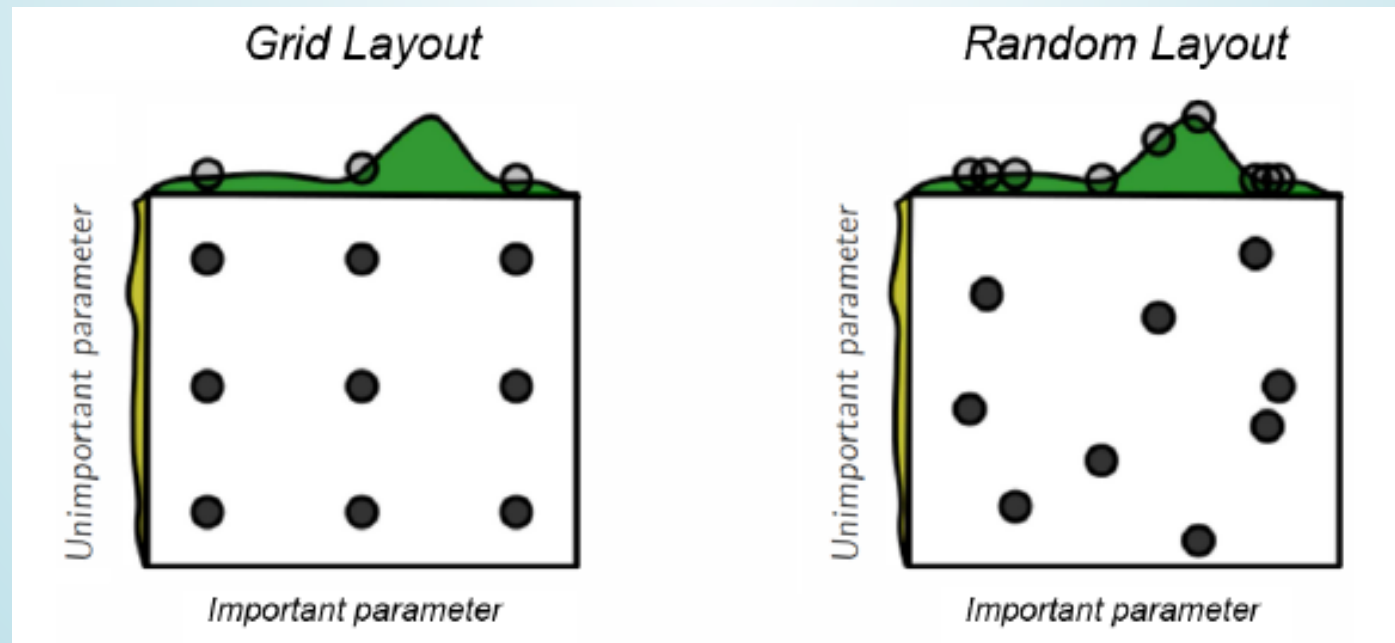
- The learning process such as it has been derived may be improved in different ways.
 - Predictions can be bad and require improvement.
 - Computations may be inefficient or slow.
 - The network may overfit and lack generalizability.
- This can be partially solved applying distinct approaches.

NETWORK ARCHITECTURE

- Network performance is affected by many hyperparameters
 - Network topology
 - Number of layers
 - Number of neurons per layer
 - Activation function(s)
 - Weights initialization procedure
 - etc.

HYPERPARAMETER TUNING

- Hyperparameters selection and tuning may be hard, due simply to dimensionality.
- Standard approaches to search for best parameters combinations are used.



HOW MANY (HIDDEN) LAYERS

- Traditionally considered that one layer may be enough
 - *Shallow Networks*
- Posterior research showed that adding more layers increases efficiency
 - Number of neurons per layer decreases exponentially
- Although there is also risk of overfitting

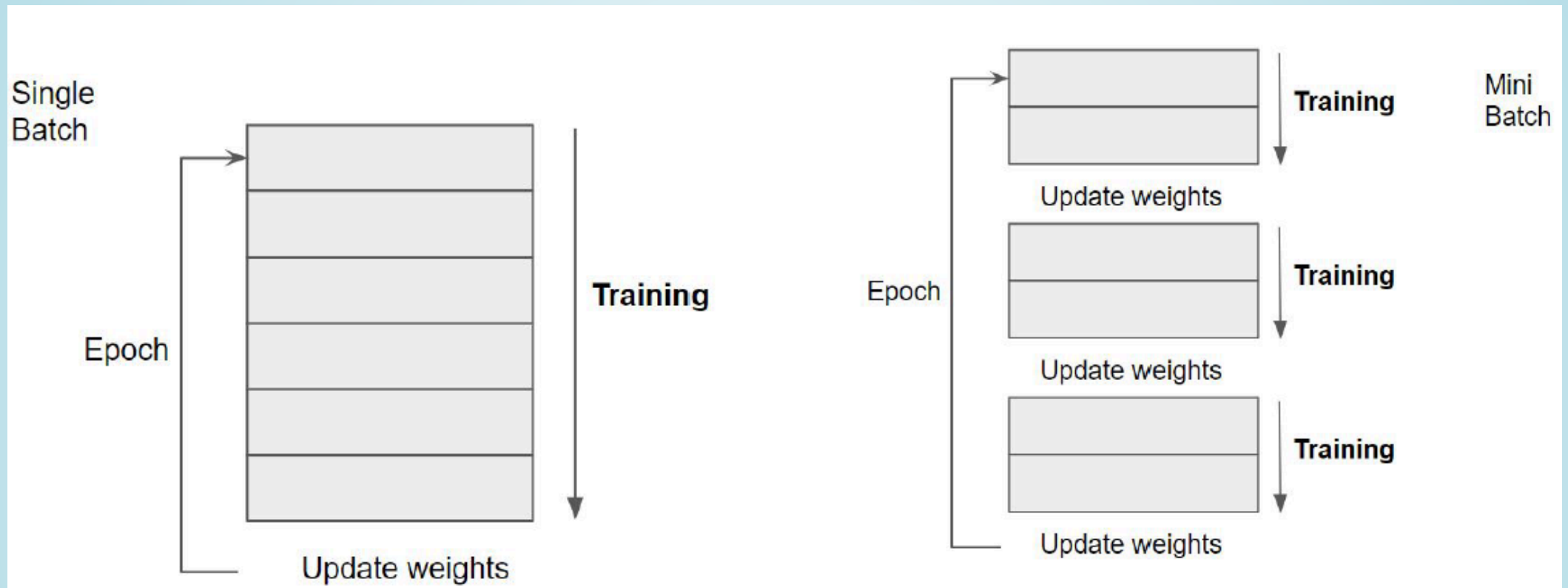
EPOCHS AND ITERATIONS

- It has been shown that using the whole training set only once may not be enough for training an ANN.
- One iteration of the training set is known as an *epoch*.
- The number of epochs N_E , defines how many times we iterate along the whole training set.
- N_E can be fixed, determined by cross-validation or left open and stop the training when it does not improve anymore.

ITERATIONS AND BATCHES

- A complementary strategy to increasing the number of epochs is decreasing the number of instances in each iteration.
- That is, the training set is broken in a number of *batches* that are trained separately.
 - Batch learning allows weights to be updated more frequently per epoch.
 - The advantage of batch learning is related to the gradient descent approach used.

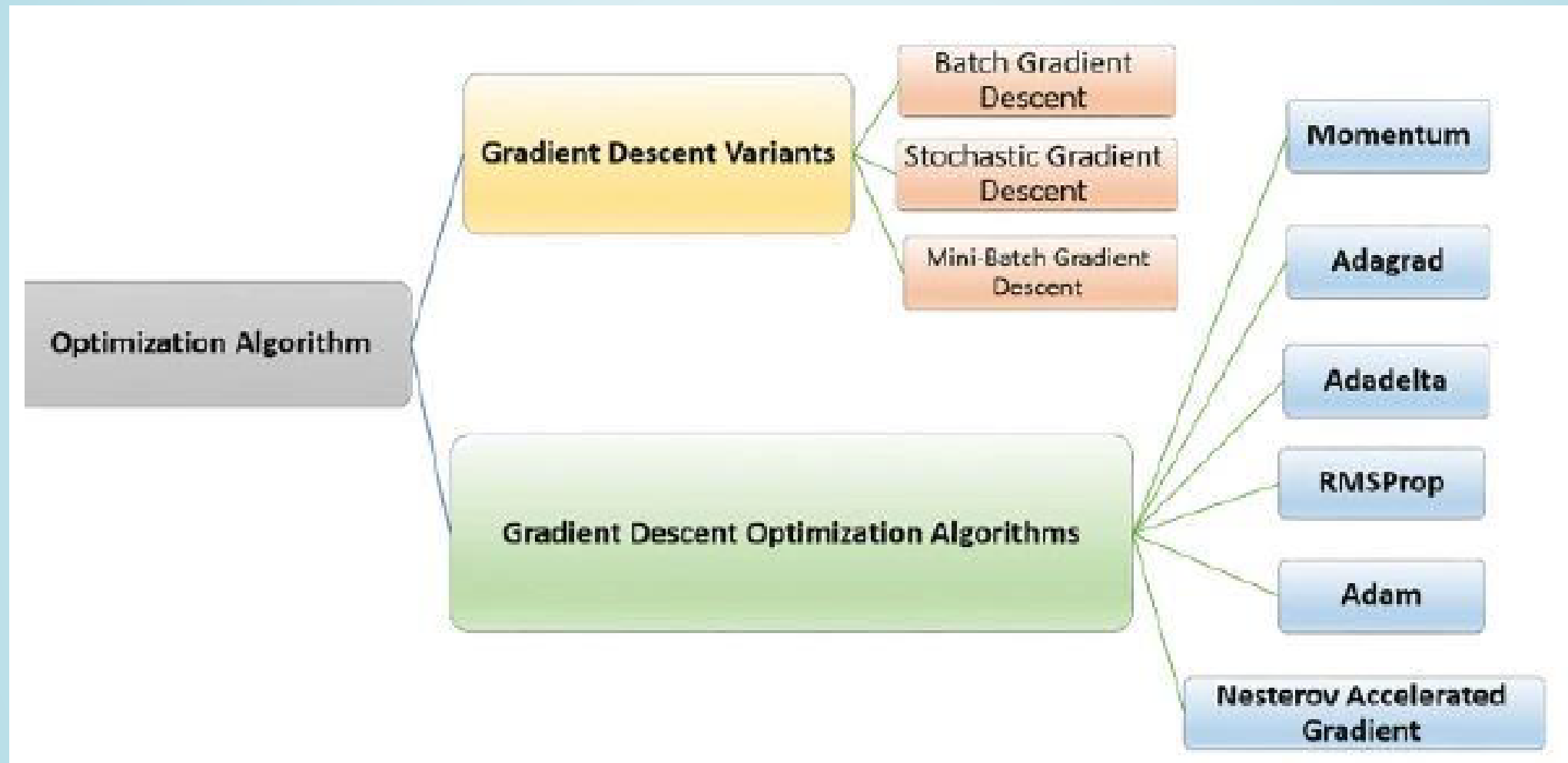
TRAINING IN BATCHES



IMPROVING GRADIENT DESCENT

- Training deep neural networks involves **millions of parameters** and **large datasets**.
- **Computing the full gradient** in every iteration is computationally expensive because:
 - It requires **summing over all training points**.
 - The cost grows **linearly with dataset size**.
- For **large-scale machine learning**, standard gradient descent becomes impractical.
- To address this, we use **alternative optimization strategies**:
 - **Gradient Descent Variants**: Control how much data is used per update.
 - **Gradient Descent Optimizers**: Improve convergence and stability.

IMPROVING GRADIENT DESCENT



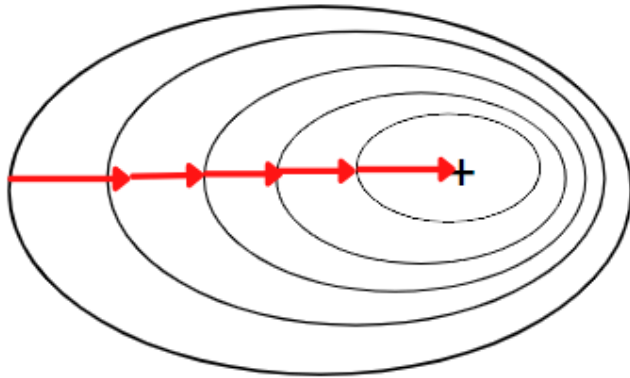
An overview of gradient descent optimization algorithms

GRADIENT DESCENT VARIANTS

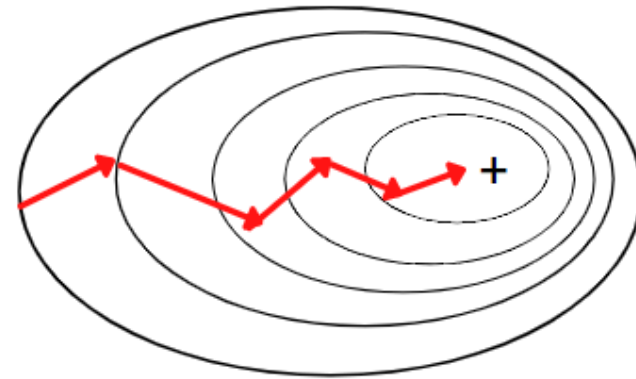
- Define **how much data is used per update**.
- Control the **trade-off between computational cost and stability**.
- Three common approaches:
 - **Batch Gradient Descent:**
 - Computes the gradient using the **entire dataset**.
 - Stable but **slow for large datasets**.
 - **Stochastic Gradient Descent (SGD):**
 - Computes the gradient using **a single training example**.
 - Faster updates but **high variance**, leading to noisy convergence.
 - **Mini-Batch Gradient Descent:**
 - Uses a **subset (mini-batch) of training data** per update.
 - Balances **speed and stability**.

GRADIENT DESCENT VARIANTS

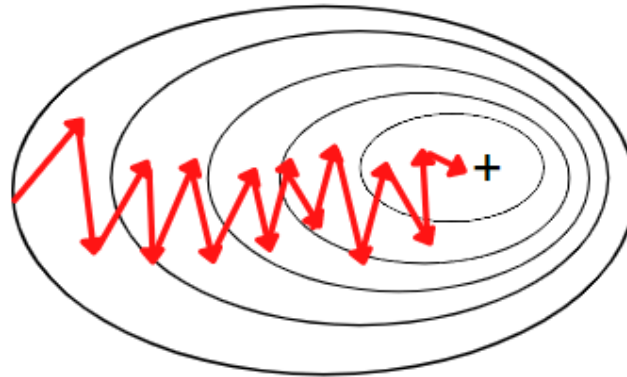
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



Source: Gradient Descent and its Types

GRADIENT DESCENT OPTIMIZERS

- Improve **learning dynamics** by modifying **how gradients are applied**.
- Help avoid **local minima, vanishing gradients, and slow convergence**.
- Common optimizers:
 - **Momentum**: Uses past gradients to accelerate convergence.
 - **Adagrad**: Adapts the learning rate per parameter.
 - **Adadelta & RMSProp**: Improve Adagrad by reducing aggressive decay.
 - **Adam**: Combines momentum and adaptive learning rates (widely used).
 - **Nesterov Accelerated Gradient**: Improves Momentum with lookahead updates.

OPTIMIZING TRAINING SPEED

- Training speed can be improved by adjusting key factors that influence convergence.
 - **Weight Initialization:** Properly initializing weights helps prevent vanishing or exploding gradients, leading to faster convergence.
 - **Adjusting Learning Rate:** A well-tuned learning rate accelerates training while avoiding instability or slow convergence.
 - **Using Efficient Cost Functions:** Choosing an appropriate loss function (e.g., cross-entropy for classification) speeds up gradient updates.

OPTIMIZING TO AVOID OVERFITTING

- Overfitting occurs when a model learns noise instead of general patterns. Common strategies to prevent it include:
 - **L2 Regularization:** Penalizes large weights to reduce model complexity and improve generalization.
 - **Early Stopping:** Stops training when validation loss starts increasing, preventing unnecessary overfitting.
 - **Dropout:** Randomly disables neurons during training to make the model more robust.
 - **Data Augmentation:** Expands the training set by applying transformations (e.g., rotations, scaling) to improve generalization.

TECHNIQUES TO IMPROVE TRAINING

Techniques	Performance Improvement	Learning Speed	Overfitting	Description
Network Architecture	X	X	X	Adjust layers, neurons and connections
Epochs, Iterations, and Batch Size		X		Controls updates per epoch to improve efficiency.
Softmax	X			Turns outputs into probabilities
Training Algorithms	X	X		GD Improvements
Learning Rate	X	X		Step size in gradient updates.
Cross-Entropy Loss		X		Optimized for classification
L2 Regularization		X	X	Penalizes large weights to prevent overfitting.
Early Stopping		X		Stops training when validation loss worsens.
Dropout	X		X	Randomly disables neurons to enhance generalization.
Data Augmentation			X	Expands training data by applying transformations.

PYTHON ANN EXAMPLE

- Lab [Lab-C3.1-IrisANN-Modular](#) contains a detailed example and a Python notebook on building a NN from scratch.
- Lab [Lab-C3.1-Dividend Prediction](#) shows how to use R to build and use a Neural network.

REFERENCES AND RESOURCES